Quickdress

Milestone 2: Sprint 1

Sprint report
November 7, 2026

Team
Aya Noaman, Malk Metwally, Nada Salem, Marwan Abouelfotuh, Omar Hassan

Project links
GitHub: https://github.com/OmarHassan4007/Quickdress
Jira: https://swe-quickdress-project.atlassian.net

Contents of this submission

Sprint 1 set out to build a working digital wardrobe: add clothes from a photo, store their details, edit and delete them, and browse them in a filtered grid (FR-01 to FR-04). This report covers the six parts of the Milestone 2 brief. Table 1 shows where each one is.

Milestone 2 item	Where	Files
1. Sprint backlog	Section 1	This document; Jira board
2. Development and demo	Section 2	The app in the GitHub repository; screenshots in `Design/App screenshots`
3. Architecture and lower-level design changes	Section 3	Design Specification, revision 2, section 7
4. Testing	Section 4	Test code in the repository; logs in `Milestone 2/Test results`
5. Sprint retrospective	Section 5	This document
6. Updated requirements and design documents with tracked changes	Section 6	`Quickdress_Requirements_Specification` and `Quickdress_Design_Specification`, revision 2

Table 1. Where each Milestone 2 item is covered.

Who did what

Each member owns a separate part of the sprint. Table 2 lists them with the evidence in GitHub.

Member	Sprint 1 work	Evidence
Omar Hassan	The `ClothingItem` model and its validation, the core contracts (`ItemStore`, `PhotoStore`), the event bus and `WardrobeManager`, as the core and plug-in packages of the microkernel.	Commits a868b5d, 813f121 (pair work with Nada)
Nada Salem	The SwiftData and photo-file stores, the wardrobe grid with its category and colour filters, the item detail screen, and the GitHub Actions workflow.	Commits a868b5d, 813f121 (pair work with Omar, pushed from Omar's account)
Aya Noaman	Editing and deleting items: the Edit item mode of the item screen, the menu on the item detail page and the delete confirmation.	Commits 9ca70f0, 59d77b4
Malk Metwally	Filtering by attributes: season, and low, medium or high warmth, formality and comfort, with the filter sheet and the removable filter chips on the Wardrobe.	Jira story "Implement filtering based on attributes"; Milestone 2 pull request
Marwan Abouelfotuh	The microkernel architecture revision, the capture flow (camera, photo library and permissions), the on-device cut-out and suggestions, the claim-ticket design across the app, the integration fix in pull request #2, the automated tests and these documents.	Pull requests #1 and #2; Milestone 2 pull request

Table 2. Sprint 1 work by team member.

1 Sprint backlog

Sprint goal: a browsable digital wardrobe, as planned in the Milestone 1 sprint plan. The stories come from the product backlog's Must requirements for wardrobe capture and browsing, plus the work that the revised architecture and the design needed. The board is kept on Jira (https://swe-quickdress-project.atlassian.net); Table 3 is the record of what was done and where to find it in GitHub.

ID	Story	Requirement	Owner	Status	Evidence
S1-01	As a user, I add a clothing item by taking a photo or choosing one from my library.	FR-01	Marwan	Done	PR #1
S1-02	The app stores each item with its category, colour, warmth, formality, comfort and seasons.	FR-02	Omar	Done	a868b5d
S1-03	Each item gets a numbered tag and can keep its exact colour.	FR-02	Marwan	Done	PR #1
S1-04	As a user, I edit or delete an item I stored.	FR-03	Aya	Done	9ca70f0, 59d77b4, fixed in PR #2
S1-05	As a user, I browse my wardrobe as a grid and filter it by category and colour.	FR-04	Nada	Done	a868b5d
S1-06	As a user, I filter my wardrobe by season and by warmth, formality and comfort.	FR-04	Malk	Done	Milestone 2 PR
S1-07	The app suggests the category, colour, ratings and seasons of a new item, on the phone (stretch goal, pulled forward).	FR-13	Marwan	Done	PR #1
S1-08	The app cuts the item out of its photo (deferred item, pulled forward).	FR-17	Marwan	Done	PR #1
S1-09	The app explains why it needs the camera before iOS asks, and still works if the camera is refused.	NFR-07	Marwan	Done	PR #1
S1-10	Core and plug-in packages for the revised microkernel architecture.	NFR-08	Marwan (design), Omar (packages)	Done	Milestone 1 (revised), a868b5d
S1-11	One visual design across the app (the claim ticket).	NFR-04	Marwan	Done	PR #1, Milestone 2 PR
S1-12	Build and test on every push and pull request.	Process	Nada	Done	a868b5d; app tests added in the Milestone 2 PR
S1-13	Tests, test report and updated specifications.	Process	Marwan	Done	This submission

Table 3. Sprint 1 backlog.

Moved to Sprint 2

Item	Why it moved
The plug-in registry	With one or two plug-ins per contract, passing them in directly was enough. The registry is needed for the weather sources, filters and recommenders of Sprint 2 (Design Specification, section 7.1).
Camera, cut-out and suggestions on a real iPhone	The iOS Simulator cannot run Apple's Vision models, and installing on a phone needs a team member's Apple ID to sign the app.
Force-quitting during a save (part of NFR-06)	Not tried in Sprint 1. Relaunch and reinstall were tested and passed.

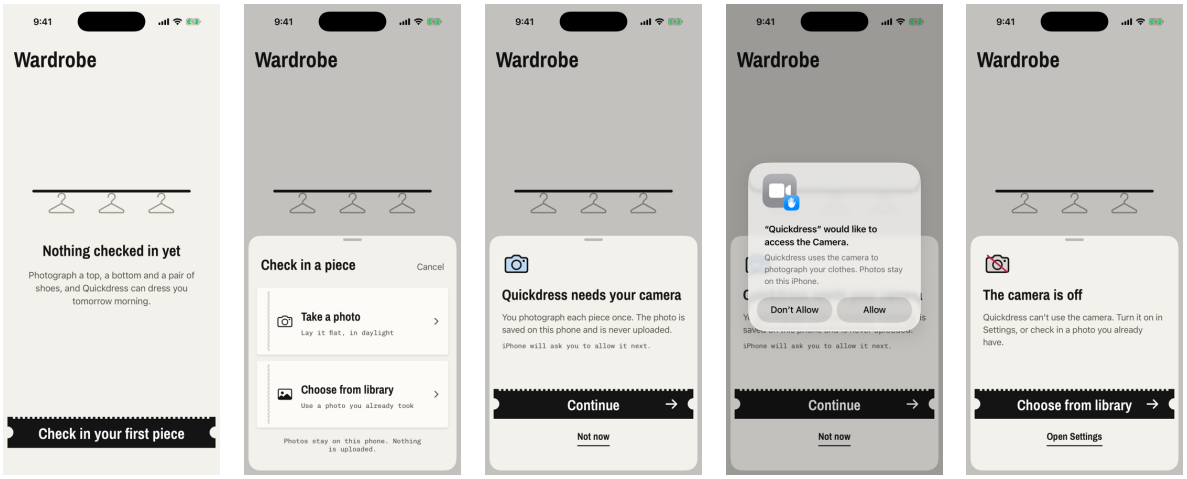
Table 4. Work moved to Sprint 2.

2 Development and demo

The app is a native iOS 17 app in SwiftUI, in the GitHub repository above. The screenshots below were taken in the iOS Simulator on an iPhone 17 Pro. Screens 09, 10 and 13 use a demo wardrobe of nine garments cut out by the app's own code; their details were checked and corrected by hand before saving, the way a user would.

2.1 Adding an item (FR-01, NFR-07)

The empty wardrobe explains what to do. Add opens a sheet with the camera and the photo library. The first time the camera is chosen, the app says why it needs it before iOS asks. If the camera was refused, the sheet says so and offers the library and Settings, so the app stays usable.

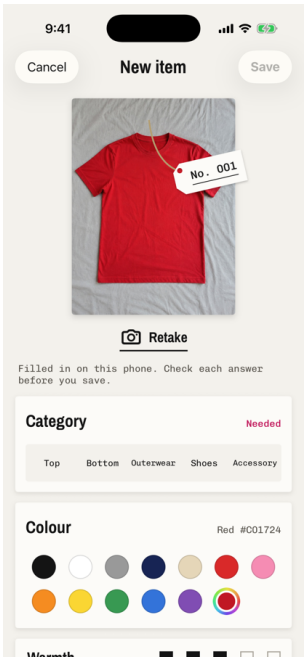


Empty wardrobe | Add sheet | Camera note | iOS asks | Camera off

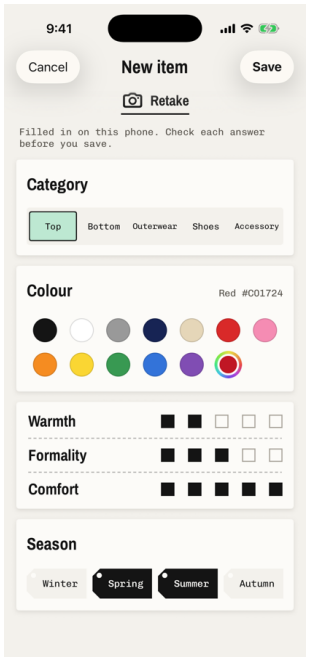
Figure 1. Adding an item.

2.2 Details, suggestions and the exact colour (FR-02, FR-13, FR-17)

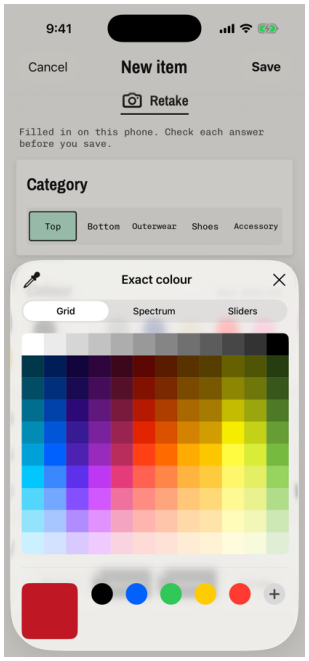
The New item screen shows the garment with its next tag number. On a phone, the garment is cut out of its background and the category, colour, ratings and seasons are filled in; the note says to check them. In the Simulator, Vision does not run, so screen 06 shows the full photo with only the colour filled in and the category still marked Needed. The colour row has the 12 palette colours and the system colour picker, whose eyedropper takes the exact colour from the photo. Save stays off until the category, colour and a season are set.



Filled in



Ready to save

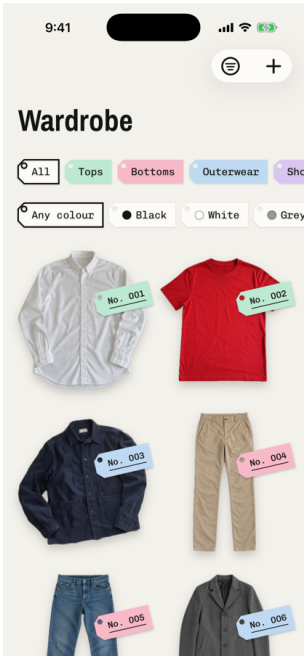


Exact colour

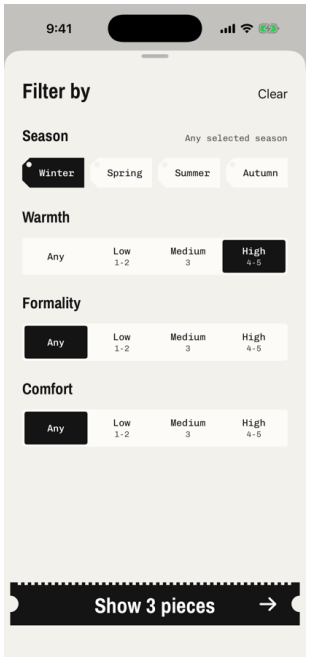
Figure 2. The New item screen.

2.3 Browsing and filtering (FR-04)

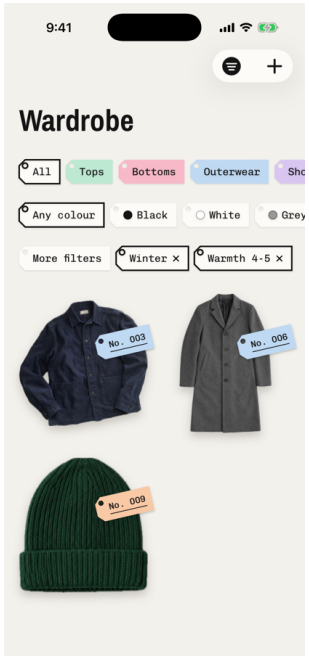
The wardrobe is a grid of garments with their numbered tags. Tag-shaped chips filter by category and colour. The filter button opens a sheet for season and for low, medium or high warmth, formality and comfort; the stub shows how many pieces match. Filters that are on appear as chips that remove themselves when tapped.



Wardrobe



Filter sheet

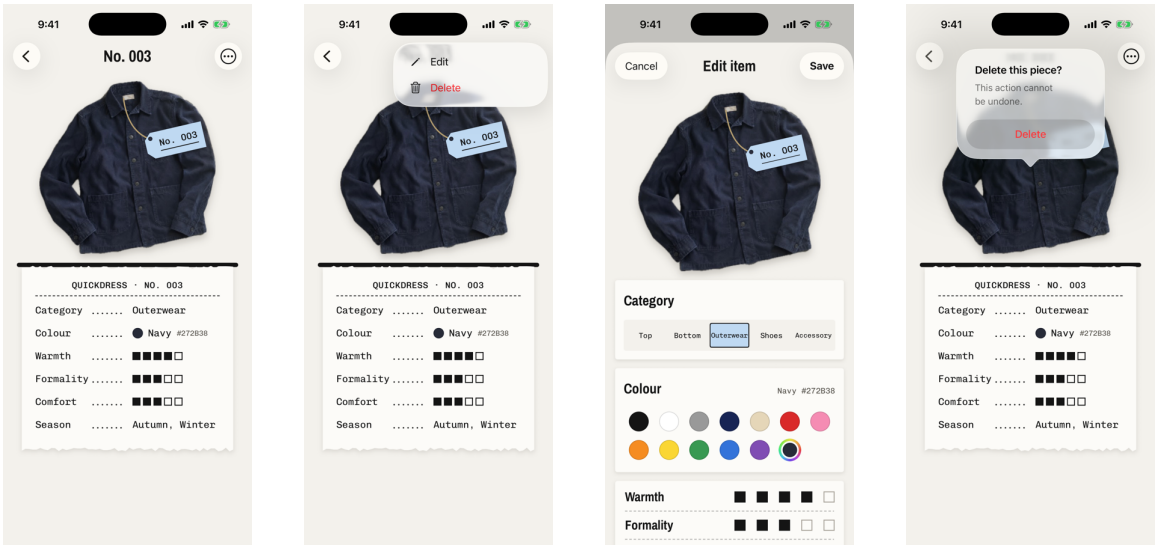


Winter, warmth 4 to 5

Figure 3. Browsing and filtering.

2.4 Detail, edit and delete (FR-03)

The detail page shows the garment and a printed receipt of its details. Its menu has Edit and Delete. Edit opens the same screen with the saved answers and without Retake, so the photo is kept. Delete asks first and then removes the item and its photo.



Detail | Menu | Edit item | Delete

Figure 4. Detail, edit and delete.

2.5 The cut-out on real photos

Because the Simulator cannot run Vision, the cut-out was run on a Mac with the app's own `SubjectLifter` code on nine garment photos: the real overshirt photo and eight generated flat-lay photos. All nine were cut out. On the white sneakers, part of the white lining was cut away with the light background (section 4.3).



Figure 5. Top: the photos. Bottom: the cut-outs the app saves.

2.6 Demo order

1. Empty wardrobe, then Add: the camera note and the iOS permission alert.
2. Choose a photo from the library; show the New item screen, change a suggestion, pick an exact colour, save.
3. The wardrobe grid: filter by category and colour, then by Winter and high warmth; remove a filter chip.

4. Open an item: Edit, change the category, save, and see the detail page update; then Delete.
5. The cut-out results on real photos (Figure 5).

3 Architecture and lower-level design changes

The architecture is the microkernel from the revised Milestone 1 design: a Foundation-only core (`QuickdressCore`) with the plug-in contracts, and plug-in packages for capture and storage. Sprint 1 kept that structure. The changes, recorded in section 7 of the Design Specification (revision 2), are:

- **Not built yet: the plug-in registry.** `QuickdressApp` creates the plug-ins and passes them in. `AddItemFlow` keeps the image sources sorted by priority. The registry comes in Sprint 2 with the weather sources, filters and recommenders.
- **Two capture helpers were added:** `SubjectLifter` for the cut-out and `ItemAnalyzer` for the suggestions. Both use Apple's Vision framework on the phone. They are helpers, not plug-ins: they implement no contract.
- **Domain changes:** `ClothingItem` has a tag `number` and an optional `exactColour`; `RGB` names exact colours by the nearest palette colour in CIE Lab; `WardrobeFilter` has seasons and a `LevelBand` for each rating.
- **Only two events exist so far,** `itemSaved` and `itemDeleted`, with the wardrobe screen's view model as their subscriber.

Figure 6 marks what Sprint 1 built on the component diagram. Figure 7 is the class diagram of the code as it is now, and Figures 8 and 9 show adding, editing and deleting an item.

Quickdress: UML component diagram (revision 2, after Sprint 1)

Microkernel architecture: a Foundation-only core with a plug-in registry and event bus; plug-ins realise the core's contracts. Arrows point from the component that depends on another to the component it uses. A green S1 badge marks what was built in Sprint 1.

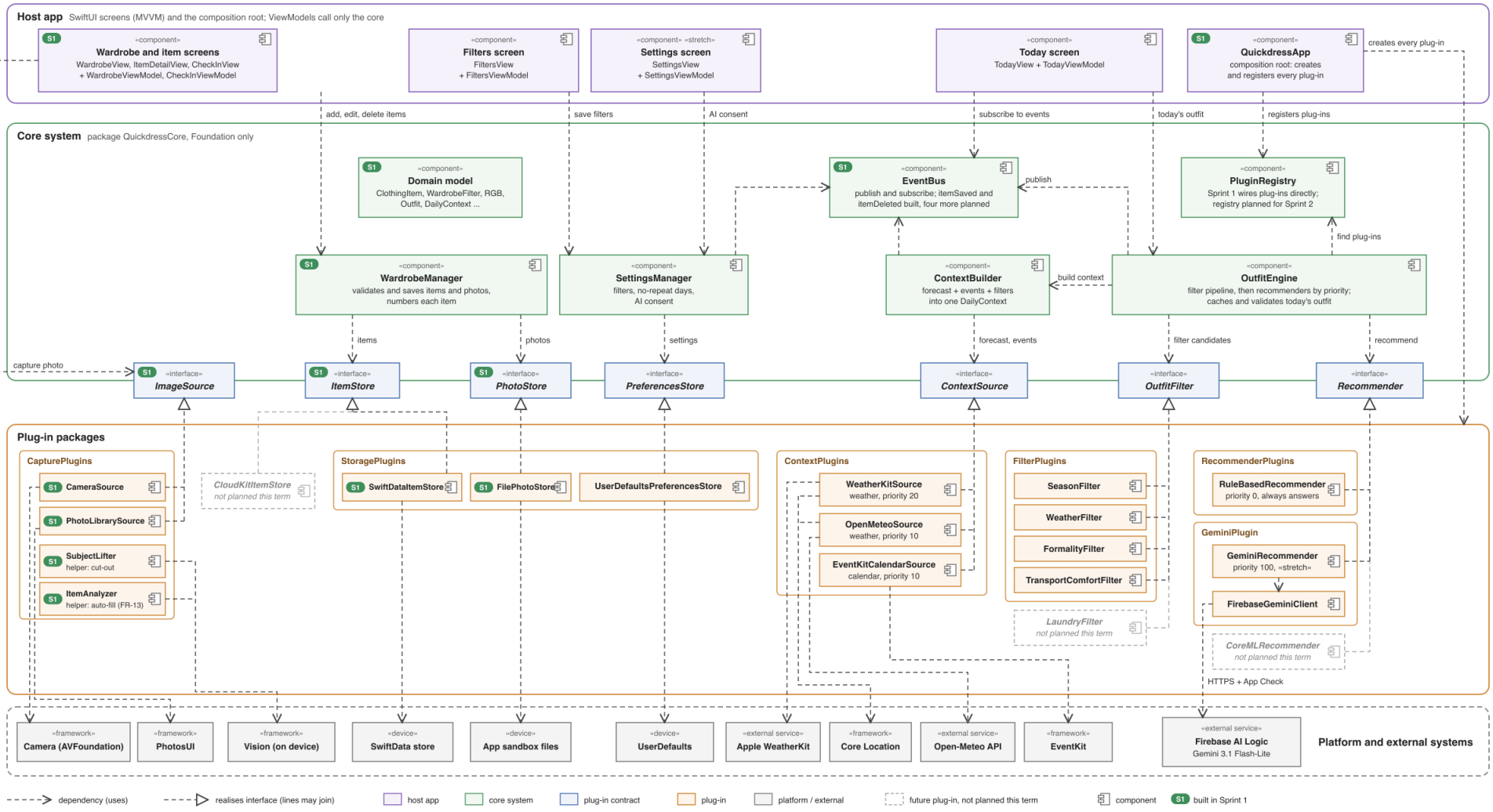


Figure 6. Component diagram, revision 2. A green S1 badge marks what Sprint 1 built.

Quickdress: UML class diagram of the code built in Sprint 1

Core services (green), plug-in contracts (orange), plug-ins and capture helpers (teal), app target (purple), value types (blue), enumerations (grey). Each frame is one Swift package or target. Views are not drawn.
Dashed arrow with open head: dependency. Dashed with hollow triangle: realises protocol. Solid with hollow triangle: protocol inherits. Solid with open head: association.

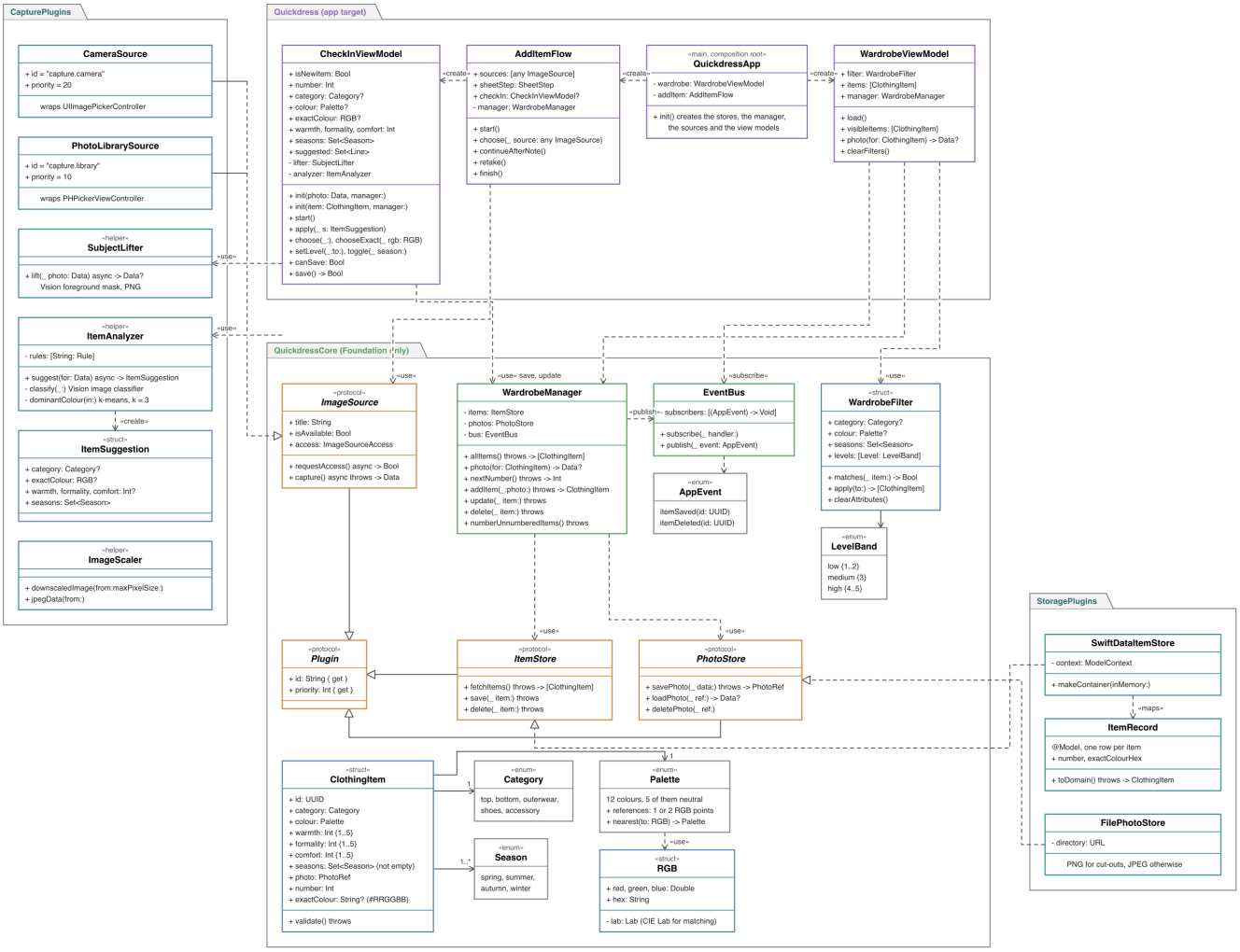


Figure 7. Class diagram of the code built in Sprint 1.

Quickdress: sequence diagram for "Add an item" (as built in Sprint 1)

Solid arrows are calls, dashed arrows are returns. Purple: screens and their models, green: core, orange: plug-ins and capture helpers.

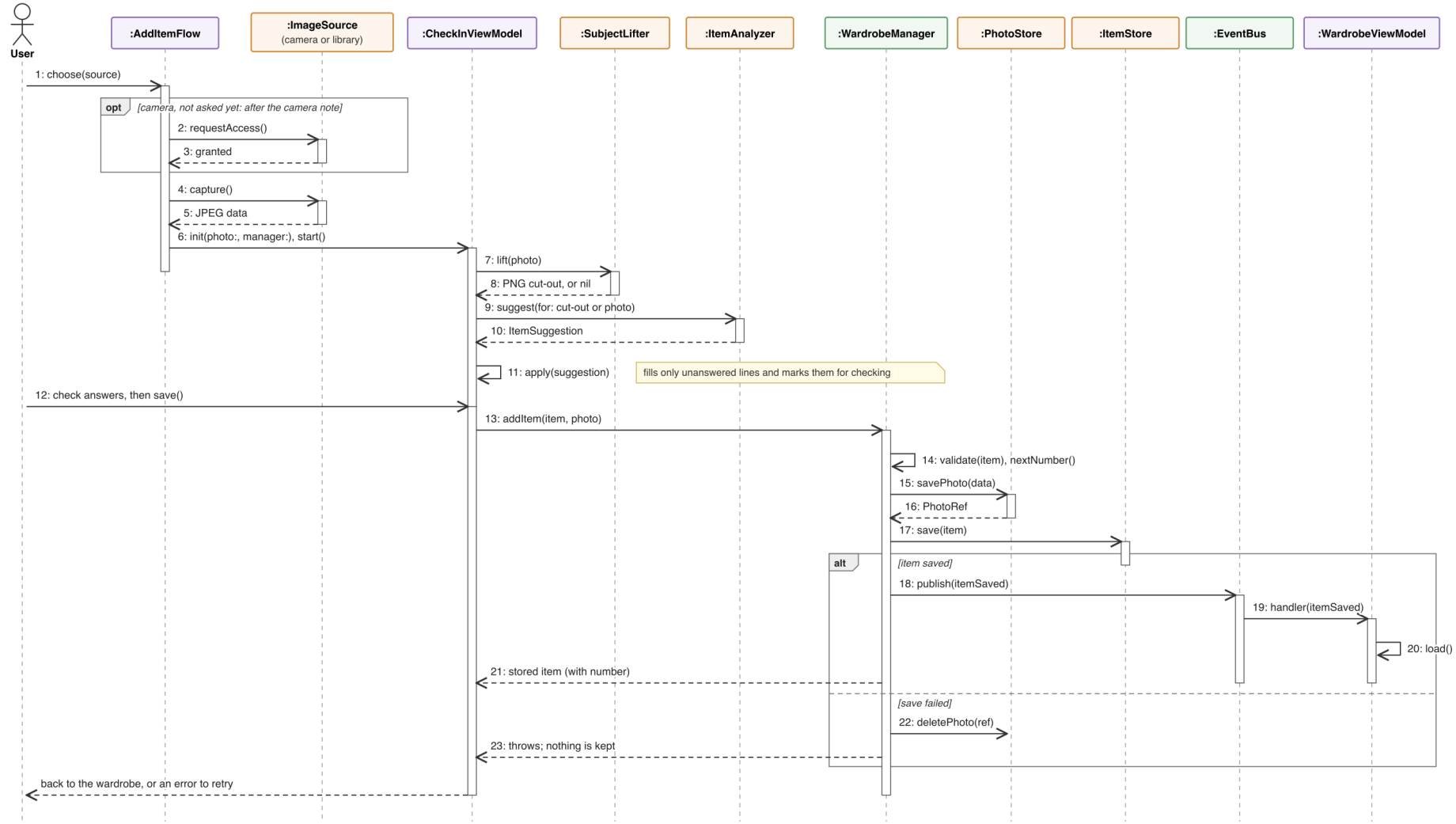


Figure 8. Sequence diagram for "Add an item".

Quickdress: sequence diagram for "Edit and delete an item" (as built in Sprint 1)

Solid arrows are calls, dashed arrows are returns. Purple: screens and their models, green: core, orange: plug-ins.

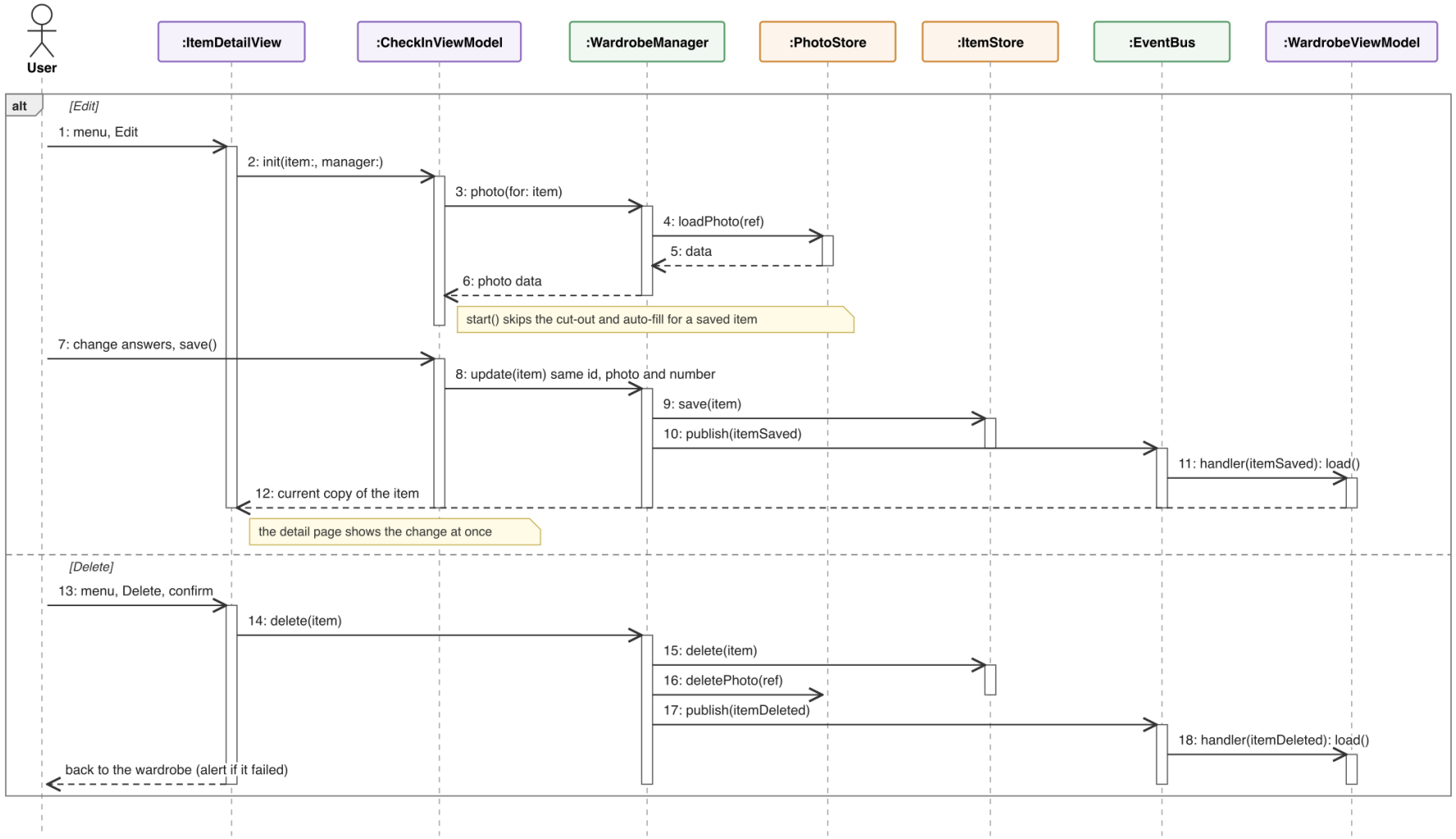


Figure 9. Sequence diagram for "Edit and delete an item".

4 Testing

4.1 Methodology

We tested at three levels and ran every automated test on each change.

- **Unit tests, white box, per component.** Each package has its own XCTest target. Components are tested alone with test doubles: in-memory versions of `ItemStore` and `PhotoStore`, one of which can be told to fail its next save. The app's view models (`CheckInViewModel`, `WardrobeViewModel`) are tested the same way, with no screen and no real storage.
- **Black-box test design for the rules.** Ratings run from 1 to 5, so we used boundary value analysis: 0, 1, 5 and 6 (and -1), and the edges of each filter band (2 and 3 for low and medium, 3 and 4 for medium and high). For filters we used equivalence partitioning: items worn in a selected season, items worn only in other seasons, and items worn in several seasons.
- **End-to-end UI tests.** XCUITest drives the real screens. The app starts with a `-uiTesting` launch argument that gives it an empty in-memory wardrobe and a test photo source, because Apple's camera and photo picker run outside the app and a test cannot drive them.
- **Manual system tests** for what automation cannot reach: the permission alerts, the real photo picker, persistence across relaunches, large text sizes, and the cut-out and suggestions on real photos, which need Vision and so a Mac or a real iPhone.
- **Continuous integration.** GitHub Actions runs the three packages' tests, builds the app and runs the app's unit tests on every push and pull request.

4.2 Results

Suite	Tests	Failures	Where it runs
QuickdressCore unit tests	40	0	Mac, `swift test`
StoragePlugins unit tests	14	0	Mac, `swift test`
CapturePlugins unit tests	7	0	Mac, `swift test`
App unit tests (view models)	17	0	iOS Simulator
End-to-end UI tests	4	0	iOS Simulator
Total automated	**82**	**0**	

Table 5. Automated test results, November 7, 2026. The logs are in `Milestone 2/Test results`.

4.3 Test cases

Table 6 lists the test cases for the Sprint 1 requirements. TC-01 to TC-15 are automated and name the test that checks them. TC-16 to TC-22 are manual.

ID	Req.	Steps	Expected	Actual	Result
TC-01	FR-01, FR-02	UI test `testAddAnItemFromAPhoto`: Add, choose the library, set Outerwear, Navy and Winter, Save, open the item.	Save is off until every answer is set; the item appears as No. 001 with those details.	As expected.	Pass
TC-02	FR-01	UI test `testCancellingANewItemKeepsNothing`: open New item, Cancel, Discard.	The wardrobe is still empty.	As expected.	Pass
TC-03	FR-03	UI test `testEditThenDeleteAnItem`: add an item, Edit it to Top, Save, then Delete and confirm.	Edit has no Retake; the detail shows Top at once; after Delete the wardrobe is empty.	As expected.	Pass
TC-04	FR-04	UI test `testFilterByCategorySeasonAndWarmth` on the nine-piece sample wardrobe.	Shoes shows only shoes; Winter shows 4 pieces; Winter and high warmth shows 1; removing the warmth chip brings back the others.	As expected.	Pass
TC-05	FR-02	Unit test `testNewItemCannotBeSavedUntilCategoryColourAndSeasonAreAnswered`.	Save stays off until category, colour and a season are set, and off again if the only season is removed.	As expected.	Pass
TC-06	FR-02	Unit tests `testLevelBoundsAreInclusive`, `testEachLevelOutsideOneToFiveIsRejected`: ratings 1 and 5, then 0, 6 and -1.	1 and 5 are valid; 0, 6 and -1 are rejected with the level named.	As expected.	Pass
TC-07	FR-02	Unit test `testItemWithNoSeasonIsRejected`.	An item with no season is rejected.	As expected.	Pass
TC-08	FR-04	Unit test `testLevelBandBoundaries`: 1, 2, 3, 4 and 5 against each band.	Low is 1 to 2, medium 3, high 4 to 5.	As expected.	Pass
TC-09	FR-04	Unit tests `testSeasonFilterKeepsItemsWornInAnySelectedSeason`, `testEveryAttributeFilterMustMatch`.	An item matches if it is worn in any selected season, and only if every filter that is set matches.	As expected.	Pass
TC-10	NFR-06	Unit tests `testFailedItemSaveDeletesThePhotoAgain`, `testAFailedSaveKeepsNothingAndShowsAnError`: the item store fails its next save.	No item and no photo are kept; the screen shows an error.	As expected.	Pass

ID	Req.	Steps	Expected	Actual	Result
TC-11	FR-03	Unit test `testSavingAnEditUpdatesTheSameItemAndKeepsItsPhotoAndNumber`.	Still one item, with the same id, photo and tag number; the photo is not stored twice.	As expected.	Pass
TC-12	FR-03	Unit tests `testDeleteRemovesTheItemAndItsPhotoAndPublishes`, `testDeletingAnItemRemovesItAndItsPhotoAndRefreshesTheGrid`.	The item and its photo are gone and the grid reloads.	As expected.	Pass
TC-13	FR-13	Unit tests `testSuggestionsNeverOverwriteWhatTheUserChose`, `testChangingASuggestedAnswerRemovesItsMark`.	A suggestion never replaces the user's answer; changing a suggested answer removes its mark.	As expected.	Pass
TC-14	FR-17	Unit test `testAGarmentPhotographedOnABedIsCutOut` (Mac): the overshirt photo.	A PNG that keeps transparency and is cropped to the garment.	As expected.	Pass
TC-15	FR-13	Unit test `testTheOvershirtIsSuggestedAsNavyOuterwear` (Mac).	Outerwear, navy, warmth 4, autumn and winter.	As expected.	Pass
TC-16	FR-13, FR-17	Run the app's `SubjectLifter` and `ItemAnalyzer` on a Mac on 9 photos: the real overshirt and 8 generated flat-lay garments.	Each item is cut out; the category and colour are right, or left for the user to fill in.	Cut-out 9 of 9 (part of the white lining of the white sneakers was lost). Category: 6 right, 1 wrong (white oxford shirt suggested as outerwear because Vision said "jacket"), 2 left empty (red t-shirt, beige chinos). Colour: 5 right at first; after fixing defect D2, 8 right (the white shirt in shade still reads grey).	Pass, with the limits noted
TC-17	NFR-07	Simulator, fresh install: Add, Take a photo, Continue, then Don't Allow, then Add again.	The app's note first, then the iOS alert; after refusing, the camera-off page with the library and Settings.	As expected (screens 03 to 05).	Pass
TC-18	FR-01	Simulator: Choose from library, pick a photo in Apple's picker.	The New item screen opens with that photo.	As expected.	Pass
TC-19	NFR-06	Simulator: save and edit items, quit and relaunch the app, then install a new build over the old one.	Every item and photo is still there, with the edit.	As expected. Force-quit during a save was not tried.	Pass (part)
TC-20	NFR-09	Simulator at the Accessibility Large text size: open the wardrobe and the New item screen.	Text grows and nothing is cut off.	The tags spilled off the photos and were cut off (defect D3). After the fix, retested: they fit.	Fail, then pass
TC-21	FR-01, FR-13, FR-17	A real iPhone: take photos with the camera and check the cut-out and suggestions.	Same results as on the Mac.	Not run: the Simulator has no Vision, and no phone was signed for testing yet.	Not run (Sprint 2)

ID	Req.	Steps	Expected	Actual	Result
TC-22	Process	Build `main` after every push.	`main` always builds.	After commits 9ca70f0 and 59d77b4, `main` did not compile (defect D1). Fixed in PR #2.	Fail, then pass

Table 6. Sprint 1 test cases.

4.4 Defects found and fixed

ID	Defect	Cause	Fix
D1	`main` stopped compiling after the edit and delete commits.	The commits called `updateItem` and `deleteItem`, which do not exist, passed a photo reference where photo data was needed, and replaced four sections of the New item screen with a comment.	PR #2: restored the sections, used `update` and `delete`, loaded the photo data, and refreshed the detail page after an edit. Merged after CI passed.
D2	Photographed denim, khaki chinos and a dark green beanie were named grey or black.	Each palette colour had one reference point, tuned to clean colours; real fabrics photograph greyer and darker.	Second reference points for blue, beige and green, and the regression test `testGarmentPhotoColoursFromSprint1Testing`. Colour went from 5 to 8 right out of 9.
D3	At large text sizes the garment tags spilled off the photos and were cut off.	The tag text followed Dynamic Type with no limit.	The tag text is capped at the default size (the tags are decorative and VoiceOver reads the number from the item's label); filter chips now grow taller instead of clipping.

Table 7. Defects found in Sprint 1 testing.

Known limits. The suggestions can be wrong or missing, which is why the screen says "Filled in on this phone. Check each answer before you save." and Save never relies on a suggestion alone. Light items on a light background can lose parts of the item in the cut-out; the user can tap "Use the original photo", or photograph light items on a darker surface.

5 Sprint retrospective

What went well

- All four Sprint 1 requirements work end to end, and two items planned for later (FR-13 and FR-17) were delivered early.
- The package split held up. The core imports nothing but Foundation, so its rules are tested in milliseconds with in-memory stores, and storage and capture each have their own tests.
- Apple's Vision framework gave the cut-out and the suggestions on the phone, with no API key and no data leaving the device.
- Testing found real problems before the demo: the build break, the wrong colour names and the clipped tags were all found and fixed in the sprint.

What could be improved

- **Build before pushing, and never push to `main` directly.** Two commits reached `main` without being built and the app stopped compiling. CI showed the failure, but nothing stopped the push.
- **A branch and a pull request for every change**, reviewed by one other member, as the design already says. Branch protection on `main` will enforce it.

- **The Simulator cannot test everything.** It has no Vision models, so the cut-out and suggestions were only tested on a Mac. We need a team iPhone signed for testing.
- **Agree the look before building screens.** For part of the sprint the New item screen used plain iOS styling while the rest of the app used the claim ticket. It was brought into line, but deciding first would have saved the rework.

Plan for Sprint 2

1. Turn on branch protection: a passing CI run and one review before merging to `main`.
2. Build the plug-in registry first, with its tests.
3. FR-05 to FR-08: the Today screen, `ContextBuilder`, `OpenMeteoSource` (and WeatherKit if a membership is available), the season and weather filters, the rule-based recommender, swap and new outfit.
4. Run the tests moved from Sprint 1 on a real iPhone: camera capture, cut-out and suggestions, and FR-13's pass mark of 24 right out of 30 real items.
5. Finish the NFR-06 test with a force-quit during a save, and run the NFR-01, NFR-03, NFR-04, NFR-05, NFR-07 and NFR-08 tests from the Design Specification.
6. Add dark mode to the claim-ticket design.

6 Updated requirements and design documents

Both specifications were updated to revision 2. Each starts with a change log that lists every revision with its date, the sections changed, what changed and why. In the body, rows shaded yellow and paragraphs that start with "Changed in rev. 2" are the changes, so a reader can see them without comparing versions. The previous versions stay in the repository: `Milestone 1` (revision 0) and `Milestone 1 (revised)` (revision 1).

Document	Main changes in revision 2
Requirements Specification	FR-02 adds the tag number and exact colour. FR-04 adds the season and rating filters. FR-13 and FR-17 are marked as delivered in Sprint 1. The sprint plan shows Sprint 1 done. Three use case descriptions match the built flows.
Design Specification	The component diagram marks what Sprint 1 built. New section 7: differences from the plan, the capture plug-ins and helpers and how the suggestions work, domain changes, sequence diagrams for adding, editing and deleting, a class diagram of the built code, the user interface design, and the tests that were run. The Sprint 1 result against its exit criteria, and new risk R12.

Table 8. Documentation updates.